COLLIDE-2V trigger proxies

User guide

Software version 0.1.0

Contents

1 Purpose and layout **1**

2 Installation and tests **2**

3 Menus and input data **2**

4 Evaluate triggers **3**

5 Override thresholds **4**

6 Acceptance and rate **4**

7 How calibration works **5**

8 Run a complete calibration **6**

9 Run individual steps or change the target rate **7**

10 Define a custom trigger **8**

11 Try an example without EOS **9**

12 Plots and reproducible runs **10**

13 Rebuild the PDF **10**

1 Purpose and layout

This package evaluates trigger selections on COLLIDE-2V Parquet events. It saves trigger decisions, measures acceptance, and adjusts transverse thresholds to approach a requested minimum-bias rate.

A trigger seed is one selection that returns pass or fail for an event. A menu is a collection of seeds. A threshold is a minimum required PT, HT, or MET.

```
Minimum-bias events -> event threshold scores -> rate versus threshold
                    -> JSON with adjusted thresholds

Events + threshold JSON -> trigger flags -> measured acceptance
```

These are proxies for CMS Phase-2 L1 triggers. Matching a COLLIDE-2V rate does not establish the same object response or signal acceptance as the CMS trigger.

Location	Contents
`src/collide2v_trigger/core/`	Trigger rules, default cuts, external definitions
`src/collide2v_trigger/calibration/`	Event scores and threshold fitting
`src/collide2v_trigger/io/`	Local and EOS Parquet access
`src/collide2v_trigger/runners/`	Command-line applications
`src/collide2v_trigger/plotting/`	Reusable plotting functions
`custom_triggers/`	Complete diphoton examples
`configs/toy_threshold_overrides.json`	Example threshold profile
`tests/`	Tests that create their own inputs

Local datasets and generated results are not required parts of the software distribution. Run commands from the repository root; replace example input paths with your own files.

2 Installation and tests

Use Python 3.10 or newer. Install local development and plotting support:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e '.[plots,dev]'
python -m unittest discover -s tests
```

Tests use temporary synthetic data and need no EOS access. For basic evaluation without plotting, `python -m pip install -e .` is sufficient. For EOS, also run:

```
python -m pip install -e '.[eos,plots]'
```

EOS needs working XRootD support and any authentication required by the storage service. Credentials are not part of this repository.

If a command is missing, repeat the editable installation with the Python in your active environment. Runners can also be executed as modules:

```
python -m collide2v_trigger.runners.calibrate_menu --help
```

3 Menus and input data

Class	Use
`ConfigurableCollide2VTriggerMenu`	Dedicated object collections; adjustable thresholds; default for evaluation and calibration
`NoIsolationCollide2VTriggerMenu`	Same rules with the module's additional electron/photon isolation cuts removed
`IDBasedTriggerMenu`	Separate particle-ID-based implementation using PUPPI candidates

The configurable menu has 42 built-in names and 33 implementable proxy selections. Nine entries require missing StaEG or tau collections. External definitions extend the configurable menu. The ID-based runner is separate and does not use this threshold-calibration interface.

Conventions in the configurable menu:

- Single-object absolute-primary-vertex DZ requirements are disabled. Relative DZ requirements remain; menu centimetres are converted to stored millimetres.

- HT is recomputed from PUPPI AK4 jets. For `PuppiHT_450`, it sums jet PT above 30 GeV within absolute eta below 2.4.
- Additional electron/photon isolation uses a maximum value of 0.15 on `IsolationVarRhoCorr` by default.
- `--uncorrected-isolation` switches the isolation branch to `IsolationVar`.
- `--no-isolation` removes these additional isolation cuts. Upstream selection of the input collections remains. Custom definitions retain their own rules.

Each Parquet row is one event. Object quantities are usually lists, with empty lists for absent objects. Matching indices in an object's PT, Eta, and Phi lists describe the same object. PT and mass are in GeV, Phi in radians, and Eta is dimensionless.

Typical collections are `L1T_MuonTight`, `L1T_Electron`, `L1T_PhotonTight`, and `L1T_JetPuppiAK4`; missing transverse momentum comes from `L1T_PUPPIMET_MET`. Inspect the exact input contract and active cuts:

```
from collide2v_trigger import ConfigurableCollide2VTriggerMenu

menu = ConfigurableCollide2VTriggerMenu()
print(menu.implementable_triggers)
print(menu.required_columns)
print(menu.thresholds_for("DoubleTkMuon_15_7"))
print(menu.target_rate_khz("DoubleTkMuon_15_7"))
```

Built-in selections currently share a menu-wide column list, even when selecting one built-in seed. It includes `L1T_ScalarHT_HT`, although trigger HT is recomputed. An external-only selection needs only its declared columns. `Event_Number` is optional.

4 Evaluate triggers

Apply the default configurable menu, or select individual seeds:

```
collide2v-apply-triggers input.parquet -o results/bits.parquet

collide2v-apply-triggers input.parquet \
  --trigger DoubleTkIsoPhoton_22_12 --no-isolation \
  -o results/diphoton_bits.parquet
```

Use the separate ID-based implementation with:

```
collide2v-apply-id-triggers input.parquet -o results/id_bits.parquet
```

Application runners accept local files and complete `root://` URLs. An EOS URL has this form:

```
root://eosproject.cern.ch//eos/project/...
```

Scoring and calibration wrappers also resolve unmounted `/eos/...` paths using their EOS endpoint.

Processing is batched. Compact outputs contain Boolean `Trigger_<name>` columns, `Trigger_passes_any`, a source-row identifier, and available identity columns such as `Event_Number`. Input provenance and menu settings are saved as Parquet metadata. Object arrays are not copied to the output.

Use `--keep-column Event_Weight` to retain an extra column explicitly. Retaining weights does not make the efficiency calculation weighted.

5 Override thresholds

Defaults are collected in `DEFAULT_THRESHOLDS` in `core/configurable_menu.py`. A partial JSON file overrides selected cuts:

```
{
  "profile": "example_only",
  "thresholds": {
    "SinglePuppiJet_230": {"jet_pt": 50.0},
    "DoubleTkMuon_15_7": {
      "leading_muon_pt": 12.0,
      "subleading_muon_pt": 5.6
    }
  }
}
```

These numbers illustrate the format; they are not calibrated recommendations. Omitted values retain defaults. Names retain the original menu numbers, so inspect `thresholds_for()` to see the active values.

Apply the included toy profile:

```
collide2v-apply-triggers input.parquet \
  --threshold-config configs/toy_threshold_overrides.json \
  -o results/toy_profile_bits.parquet
```

Python also accepts individual overrides:

```
from collide2v_trigger import ConfigurableCollide2VTriggerMenu

menu = ConfigurableCollide2VTriggerMenu(
    threshold_config="configs/toy_threshold_overrides.json",
    threshold_overrides={"SinglePuppiJet_230": {"jet_pt": 60.0}},
)
event = {
    "L1T_JetPuppiAK4_PT": [65.0],
    "L1T_JetPuppiAK4_Eta": [0.5],
}
print(menu.evaluate("SinglePuppiJet_230", event))  # True
```

Constructor overrides take priority over JSON, which takes priority over defaults. This direct evaluation only needs fields used by that rule; file runners enforce their declared input contract.

6 Acceptance and rate

Efficiency is the fraction of all events that pass:

$$\text{efficiency} = \frac{\text{passing events}}{\text{total events}}.$$

At a collision rate of 40 MHz:

$$\text{rate in kHz} = 40{,}000 \times \text{efficiency}.$$

For example, 500 passes in 1,000,000 events give efficiency 0.0005, or 0.05%, and rate 20 kHz. Use the fraction, rather than the percentage, in the equation.

Combine trigger-bit files and plot their efficiencies:

```
collide2v-trigger-efficiencies 'results/bits_*.parquet' \
  -o results/efficiencies.txt
python scripts/plotting/plot_trigger_efficiencies.py \
  results/efficiencies.txt -o results/efficiencies.png
```

Counts are summed before division. The calculation is unweighted; empty-object events stay in the denominator and null flags count as failures. Files must have matching Boolean trigger columns. Use disjoint events and compatible menu settings; repeated events in different files count again.

On a signal sample such as ggH to two photons, the fraction is **signal acceptance**. Multiplying it by 40 MHz gives a 40 MHz-equivalent rate. A physical signal rate also needs the production rate and any sample-filter acceptance. Minimum-bias samples must represent the intended collision population for the same normalization to describe a background rate.

`Trigger_passes_any` measures the OR of the evaluated seeds. Individual seed rates overlap and must not be added to estimate the total menu rate.

7 How calibration works

7.1 Move several cuts together

Calibration varies one reference cut in GeV. Other selected transverse cuts move in the same proportion, preserving the base-profile ratios:

$$\text{new leg cut} = \text{base leg cut} \times \frac{\text{new reference cut}}{\text{base reference cut}}.$$

For `DoubleTkMuon_15_7`, the reference is the leading-muon cut:

Reference cut	Leading-muon cut	Subleading-muon cut
15 GeV	15 GeV	7 GeV
30 GeV	30 GeV	14 GeV
45 GeV	45 GeV	21 GeV

For built-in seeds, the largest positive scanned cut is the default reference; this can be HT or MET. The jet floor `ht_jet_pt_min` stays fixed. Eta, mass windows, isolation, relative DZ, and other fixed rules also stay fixed.

7.2 Save one boundary score per event

The scanner calls the trigger rule while bisecting the reference-cut range. It finds the largest cut confirmed to pass. Increasing the scanned cuts must never make a failing event pass: this is the monotonicity requirement.

Output has one column, `pt_threshold_gev`, in the original row order:

Saved value	Meaning
Finite value below upper bound	Largest cut confirmed to pass; boundary lies within score tolerance above it
`NaN`	Event already fails at the lower search bound
Value equal to upper bound	Event still passes there; its boundary may be higher

The score is a **threshold coordinate**, not necessarily an object's PT. Metadata records its trigger, reference field, scaled fields, base cuts, bounds, precision, source, and relevant custom-definition information.

Within the saved bounds, a rate scan counts scores at or above the proposed cut. NaN scores never pass but remain in the total count. Boundary approximations can differ from direct evaluation within the scoring tolerance. Check final cuts on independent events where possible. Every proposed cut reuses the same scores; separate trigger-bit files are unnecessary.

7.3 Fit the rate and interpret precision

Bisection narrows a threshold interval until it reaches `--tolerance` in GeV. A monotonic scan evaluates a regular grid with `--spacing` in GeV. The default `at_most` policy selects the lowest evaluated cut with rate at most the target. `closest` selects the nearest measured rate and may exceed it.

Menu-workflow setting	Job
`--score-tolerance`	Precision of each saved event boundary
`--tolerance`	Precision of the final rate-bisection search
`--spacing`	Separation between regular rate-curve points

A smaller final-search tolerance does not recover information lost to coarse scores. Rate-reporting tolerances flag poor matches; they do not change the search stopping condition.

Finite statistics and tied scores make the rate a staircase. At 40 MHz:

$$\text{one-event rate step in kHz} = \frac{40{,}000}{\text{total events}}.$$

For 100,000 events the step is 0.4 kHz; for 1,000,000 it is 0.04 kHz. Ties can cause larger jumps. A precise threshold need not achieve the target rate.

8 Run a complete calibration

The top-level runner selects common files, saves scores, performs bisection, makes a regular rate curve, and exports a combined threshold JSON.

```
collide2v-calibrate-menu /path/to/minbias \
  --trigger SinglePuppiJet_230 --n-files 10 \
  --score-tolerance 0.25 --tolerance 0.25 --spacing 1 \
  -o results/jet_calibration
```

Use a local directory, a `root://` directory, or an unmounted `/eos/...` path. For the known production directory and all implementable seeds:

```
MINBIAS=/eos/project/f/foundational-model-dataset/samples/production_final/minbias
collide2v-calibrate-menu "$MINBIAS" --n-files 100 \
  --score-tolerance 0.25 --tolerance 0.25 --spacing 1 \
  -o results/menu_calibration
```

Defaults are ten files, random selection with seed 12345, a 0–1000 GeV range, and 40 MHz. Requested file count is capped at availability. Preserve the manifest: a random seed alone does not fix a changing directory listing.

```
results/menu_calibration/
  input_manifest.json
  calibration_summary.json
  calibrated_thresholds.json
  SinglePuppiJet_230/
    scores/<source>_SinglePuppiJet_230_pt_threshold.parquet
    bisection.json
    bisection_points.txt
    monotonic.json
    monotonic_points.txt
    monotonic_points.png
```

The export uses bisection; a coarse regular grid may choose a different point. Apply the combined profile with:

```
collide2v-apply-triggers input.parquet \
  --threshold-config results/menu_calibration/calibrated_thresholds.json \
  -o results/calibrated_bits.parquet
```

Repeat a calibration command to resume using compatible saved scores. Use a new output directory for changes to sample, rules, base cuts, or scoring settings. Curve spacing and final-search settings can change while reusing scores. `--no-plots` skips image generation.

Read each seed's status in `calibration_summary.json`:

Status	Meaning
`calibrated`	Target lies within endpoint rate range and a cut was selected; inspect rate agreement and quality flags too
`unreachable`	Target lies outside endpoint rate range
`failed`	Processing encountered an error; inspect its reason

Unreachable seeds retain base thresholds by default. With `--export-unreachable-boundary`, they export the selected boundary instead. Status stays unreachable, `calibration_complete` stays false, and quality flags remain. Boundary export does not make the target reachable.

Defaults flag fewer than 25 passing events and rate differences above the larger of 10% of the target or one event's rate step. Other flags identify zero acceptance and boundary cases. Zero-rate targets are always flagged for review. If no event passes at the lower bound, the sample cannot determine a preferred cut. A zero threshold does not disable a seed.

Inspect measured rates, applied cuts, and flags. A combined profile can mix calibrated cuts, retained base cuts, and explicitly exported boundary cuts.

9 Run individual steps or change the target rate

Save scores for one trigger over ten files:

```
collide2v-scan-eos-directory /path/to/minbias SinglePuppiJet_230 \
  --n-files 10 --tolerance 0.25 -o results/jet_scores
```

Here `--tolerance` is the per-event precision. The single-file alternative is:

```
collide2v-scan-thresholds input.parquet SinglePuppiJet_230 \
  -o scores.parquet
```

Fit scores to a custom 25 kHz target, then make a regular curve:

```
collide2v-calibrate-threshold results/jet_scores \
  --method bisection --target-rate-khz 25 --tolerance 0.25 \
  --scan-points results/jet_scores/bisection_points.txt

collide2v-calibrate-threshold results/jet_scores \
  --method monotonic --spacing 1 --target-rate-khz 25 \
  --scan-points results/jet_scores/monotonic_points.txt
collide2v-plot-threshold-scan results/jet_scores/monotonic_points.txt
```

Omit `--target-rate-khz` to use the recorded menu target. This single-seed command supports a custom target; the complete-menu CLI takes targets from the menu/custom definitions.

Fits print and save cut, efficiency, and rate. Scan tables contain those quantities, counts, and metadata, so plotting needs no original events. A single-seed fit JSON is a report; use the complete-menu export for a directly loadable combined threshold profile.

10 Define a custom trigger

Keep the rule and metadata in one Python file exporting a `TriggerDefinition` as `TRIGGER`, or a list/tuple as `TRIGGERS`. Included examples:

File in `custom_triggers/`	Fixed mass window	Default cut per photon	Target
`diphoton_mass_50_200.py`	50–200 GeV	1 GeV	20 kHz
`diphoton_mass_120_130.py`	120–130 GeV	1 GeV	10 kHz

They test all distinct PhotonTight pairs. Both photons must pass a common PT cut and the pair must satisfy the inclusive mass window. There are no extra eta or isolation cuts; upstream collection selection still applies.

For effectively massless photons:

$$m_{\gamma\gamma}^2 = 2p_{T1}p_{T2}\left[\cosh(\eta_1 - \eta_2) - \cos(\phi_1 - \phi_2)\right].$$

The two PT values are transverse momenta; eta and phi describe direction. PT in GeV gives mass in GeV. Equal-eta photons half a turn apart have mass $2\sqrt{p_{T1}p_{T2}}$.

A definition includes its name/version, required columns, default cuts, target rate, fixed parameters, scaled fields, and reference field. Its `evaluate(event, thresholds, fixed_parameters)` function returns a Boolean. Use the example files as templates.

Evaluation must be deterministic. Calibration additionally requires `monotonic=True`: increasing the scanned cuts cannot turn failure into success. This is the author's promise, not a property the framework can prove.

Calibrate the wide-window example and apply its exported profile:

```
collide2v-calibrate-menu /path/to/minbias \
  --custom-trigger-file custom_triggers/diphoton_mass_50_200.py \
  --trigger DoublePhoton_Mass50_200 --no-isolation --n-files 10 \
  --score-tolerance 0.01 --tolerance 0.01 --spacing 0.25 \
  -o results/custom_calibration

collide2v-apply-triggers input.parquet \
  --custom-trigger-file custom_triggers/diphoton_mass_50_200.py \
  --trigger DoublePhoton_Mass50_200 --no-isolation \
  --threshold-config results/custom_calibration/calibrated_thresholds.json \
  -o results/custom_bits.parquet
```

Use the same definition, variant, and isolation settings when applying a profile. Scores and profiles record a SHA-256 content hash: editing even comments changes the identity and requires new compatible scores/profiles. Moving an unchanged file preserves its identity. A fixed mass window can make a target unreachable even with a zero PT cut.

11 Try an example without EOS

After installation, run this from the repository root. It creates diphoton masses 125, 60, and 20 GeV, followed by an empty event:

```
mkdir -p results/demo
python - <<'PY'
from math import pi
import pyarrow as pa
import pyarrow.parquet as pq

events = pa.table({
    "Event_Number": [1, 2, 3, 4],
    "L1T_PhotonTight_PT": [[62.5, 62.5], [30., 30.], [10., 10.], []],
    "L1T_PhotonTight_Eta": [[0., 0.], [0., 0.], [0., 0.], []],
    "L1T_PhotonTight_Phi": [[0., pi], [0., pi], [0., pi], []],
})
pq.write_table(events, "results/demo/events.parquet")
PY

collide2v-apply-triggers results/demo/events.parquet \
  --custom-trigger-file custom_triggers/diphoton_mass_50_200.py \
  --trigger DoublePhoton_Mass50_200 -o results/demo/bits.parquet
collide2v-trigger-efficiencies results/demo/bits.parquet \
  -o results/demo/efficiencies.txt
```

The flags should be `True, True, False, False`: two of four events pass, giving efficiency 50%. To demonstrate calibration:

```
collide2v-scan-thresholds results/demo/events.parquet \
  DoublePhoton_Mass50_200 \
  --custom-trigger-file custom_triggers/diphoton_mass_50_200.py \
  --tolerance 0.25 -o results/demo/scores/events.parquet
collide2v-calibrate-threshold results/demo/scores \
  --method bisection --target-rate-khz 10000 \
  --scan-points results/demo/scores/points.txt
```

The artificial target is **10,000 kHz**, or one of four events at 40 MHz. The cut separates the 30 GeV pair from the 62.5 GeV pair. This tests software behavior with deliberately low statistics; it is not a physics rate measurement. Use a fresh output directory to repeat the example because application

runners refuse existing outputs.

12 Plots and reproducible runs

Maintained scripts under `scripts/plotting/`:

- `plot_trigger_efficiencies.py`: measured trigger-bit efficiencies.
- `plot_trigger_seed_rates.py`: reference rates stored in menu metadata.
- `plot_leading_object_distributions.py`: leading-object spectra and simple single-variable rate thresholds.
- `plot_diphoton_mass.py`: pair masses and mass-window event acceptance.

For example:

```
python scripts/plotting/plot_diphoton_mass.py /path/to/minbias \
  --n-files 100 -o results/diphoton_mass.png
```

The mass plot uses all distinct pairs by default. Histogram entries count pairs, while event acceptance counts each event once. It does not apply calibrated asymmetric diphoton cuts. Use `--help` for detailed options.

Keep the code revision, dependency versions, custom files, base profile, full command, selected-file manifest, and generated summary/profile in your analysis records. Include empty events, use disjoint inputs, and check fitted cuts on separate minimum-bias events. Statistical flags assume independent events; file correlations and repeated pileup need separate checks.

See the physics reference guide for sources and original menu-label conventions. Stored defaults and target rates are the values used by this code. The toy profile is illustrative; a calibrated profile applies to its dataset, selection, and normalization.

13 Rebuild the PDF

The Markdown is the source for the matching PDF. Install Pandoc, pdfLaTeX, and the LaTeX `fvextra` and `lmodern` packages, then run:

```
bash docs/build_user_guide.sh
```

These document-generation tools are separate from the Python dependencies.